

Are Traditional Lakehouse Architectures Still Relevant in the AI Era?

Executive summary

Traditional lakehouse architectures (object storage + open table formats like Delta Lake / Iceberg / Hudi + scalable compute + a catalog/governance layer) remain highly relevant in the AI era, but their role is shifting from “the one platform that does everything” to “the durable, cost-effective system of record for analytics + ML,” surrounded by specialized serving layers (feature stores, vector search, streaming/state stores, semantic layers) that meet AI-driven latency and access-pattern requirements. The lakehouse thesis—unifying the flexibility and low cost of data lakes with the reliability and performance features of warehouses—still holds, and is reinforced by the industry’s move toward open table formats and cross-engine interoperability. ¹

What *has* changed in the AI era is the “center of gravity” of data access: RAG-style retrieval, embeddings, and hybrid search have made vector indexes and unstructured/semi-structured content first-class citizens, while NL2SQL and AI coding agents increase both the volume and variability of queries and code changes. This raises the importance of strong governance, lineage, and least-privilege access controls (to prevent data leakage and control cost) and pushes many teams toward architectures that pair a lakehouse (system of record) with managed serving systems for AI access patterns. ²

A pragmatic conclusion for most engineering orgs in 2026: keep (or adopt) a lakehouse *core* if you need open, scalable, multi-workload analytical/ML data, but modernize it with (a) open catalogs and strong metadata, (b) streaming/CDC ingestion, (c) a semantic layer for consistent metrics and safer NL2SQL, and (d) a vector retrieval strategy (warehouse-native vector search for simplicity and cost; dedicated vector DB when you need high-QPS, low-latency, multi-tenant retrieval at large scale). ³

Definitions and what “the AI era” changes

A **data lake** is typically low-cost object storage (S3/ADLS/GCS) holding files in open formats; it’s flexible, but historically lacked “database-like” guarantees (transactions, consistent schema evolution, performant SQL semantics). **Open table formats** (Delta Lake, Apache Iceberg, Apache Hudi) add a metadata/transaction layer on top of columnar files (commonly Parquet), enabling database-like table behavior—ACID-ish transactions, schema evolution, snapshots/time travel, and better performance optimizations—without giving up open storage. ⁴

A **data warehouse** is an analytical database optimized for SQL analytics, performance, and governance. Modern cloud warehouses often separate compute and storage and remove infrastructure management from customers; BigQuery emphasizes fully managed serverless warehousing and separate innovation of storage vs compute. ⁵

A **lakehouse** is an architectural pattern aiming to unify lake flexibility/cost with warehouse reliability and performance, often by using open table formats + scalable compute + catalog/governance. The CIDR lakehouse paper frames this as unifying data warehousing and advanced analytics on open storage, and argues it addresses issues like staleness, lock-in, and limited use-case support. ⁶

In the **AI era**, three data-access patterns become disproportionately important:

Natural-language and agent-driven querying: Tools like Gemini in BigQuery can generate/explain/fix SQL, which lowers the barrier to querying but increases the need for strong metadata and governance to avoid incorrect queries, sensitive data exposure, and runaway costs. ⁷

Retrieval-Augmented Generation (RAG): RAG retrieves relevant external knowledge before generating answers; surveys emphasize it helps with hallucination/out-of-date knowledge, but it also introduces new pipelines (chunking, embedding, indexing) and security risks (prompt injection through retrieved content). ⁸

Vector and hybrid search: Warehouses and lakehouse platforms increasingly offer native vector types and search functions (Snowflake VECTOR + similarity functions; BigQuery VECTOR_SEARCH; Databricks Mosaic AI Vector Search; Snowflake Cortex Search as a managed retrieval service), while dedicated vector databases (Pinecone, Milvus) remain important for certain latency/scale profiles. ⁹

Part one: Lakehouses explained like you're a junior engineer

The simplest mental model

Imagine your company has a huge **shared folder** (object storage) where everyone dumps files: CSVs, JSON, logs, exports. That's a **data lake**. It's cheap and flexible, but it's also messy: two people might update the same dataset at once, someone might change a schema silently, and it can be hard to know what's trustworthy.

A **lakehouse** upgrades that shared folder so it behaves more like a **real database**—without forcing you to move all data into one proprietary database system. It does this using:

Columnar files (usually Parquet) for efficient analytics reads. ¹⁰

A table format (Delta Lake / Iceberg / Hudi) that adds “database rules” on top of those files—transactions, schema evolution, version history/time travel, and query optimizations. ¹¹

A compute engine (Spark, Trino, or a managed platform like Databricks) that runs SQL and transforms data. Spark Structured Streaming also lets you treat streaming computations similarly to batch, which matters for near-real-time pipelines. ¹²

A catalog/governance layer (a “data directory”): Unity Catalog is explicitly described as unified data and AI governance built into Databricks, and it can capture runtime lineage down to column level. ¹³

Core components of a traditional lakehouse

“Traditional” lakehouse usually means a few layers working together:

Storage layer: object storage + Parquet files. ¹⁰

Table layer: Delta/Iceberg/Hudi table metadata and transaction semantics. ¹⁴

Compute/query layer: Spark/Trino/warehouse engines that read the tables. Trino's lakehouse connector is explicitly designed to query/write Iceberg, Delta Lake, Hudi, and Hive tables through one connector. ¹⁵

Catalog + governance + lineage: a centralized "what exists, who can access it, where it came from" system (Unity Catalog, OpenLineage-style lineage standard, etc.). ¹⁶

Ingestion + transformation pipelines: batch loads + CDC/streaming + ETL/ELT jobs. ¹⁷

Many lakehouse implementations use a **medallion architecture** (often called bronze/silver/gold): raw ingested data → cleaned/standardized → curated analytics tables. (This is commonly described in lakehouse reference architectures and operational guidance.) ¹⁸

What it's good at

A lakehouse is good when you need:

One durable place for analytics + ML training data: The lakehouse concept was explicitly pitched as unifying data warehousing and advanced analytics on open storage. ⁶

Open data & interoperability: Iceberg is designed as an open table format that works across multiple engines (Spark, Trino, Flink, etc.). ¹⁹

Reliable pipelines with rollback/history: Databricks documents that each modifying operation creates a new table version and that history/time travel supports auditing and rollback use cases. ²⁰

Lower storage cost vs "all-in warehouse": object storage + open formats typically reduce storage cost, while compute is scaled on demand (the general architectural driver behind lakehouse adoption). ²¹

Where it struggles (especially for AI apps)

Lakehouses are not automatically great at:

Low-latency "serving" workloads: Object storage + distributed SQL is usually fine for analytics, but OLTP-style latencies often require separate databases or caches. (Many modern reference architectures add an operational serving store alongside the lakehouse.) ²²

Vector retrieval at high QPS + low latency: Some platforms add native vector search (BigQuery VECTOR_SEARCH; Snowflake VECTOR functions; Databricks vector search; Snowflake Cortex Search), but dedicated vector DBs still often win when you need tight latency SLOs, multi-tenancy isolation, and "search-first" operational behavior. ²³

Keeping humans and AI agents "on the rails": NL2SQL and agentic systems increase the chance of wrong queries, exposed sensitive data, and costly query patterns. This elevates the importance of semantic layers (consistent metric definitions) and governance/lineage. dbt's Semantic Layer is explicitly intended to centralize metric definitions and handle joins automatically. ²⁴

How AI coding agents change the picture in plain terms

AI coding agents (example: Claude Code) can read a codebase, make changes across files, run tests, and deliver committed code—so they can help build and iterate on data pipelines faster. ²⁵

But speed creates a new problem: more pipelines, more queries, more “shadow data products.” That makes good catalogs, lineage, and access controls *more* important, not less. Unity Catalog’s positioning explicitly emphasizes unified governance for “data and AI assets,” and it captures runtime lineage across queries (including column-level). ¹³

Also, AI agents and LLM apps introduce new security failure modes (prompt injection). The UK NCSC warns prompt injection should be viewed as exploitation of an “inherently confusable deputy,” and OWASP’s Top 10 for LLM Apps highlights prompt injection as a major category and notes that even RAG and fine-tuning do not fully mitigate it. ²⁶

A picture of a modern lakehouse-centered architecture

```

flowchart LR
    subgraph Sources
        A[Legacy Relational DB] -->|CDC| I
        B[MongoDB / Document DB] -->|Change streams / CDC| I
        C[Logs / Events] -->|Streaming| I
        D[Files: CSV/JSON/PDF] -->|Batch| I
    end

    I[Ingestion Layer<br/>CDC + Batch + Streaming] --> BR[Bronze: Raw tables<br/>OTF on Parquet]
    BR --> SI[Silver: Cleaned/standardized]
    SI --> GO[Gold: Curated marts<br/>Star schema / wide tables]

    GO --> BI[BI / Analytics SQL]
    GO --> ML[Model training + batch inference]
    SI --> FS[Feature Store<br/>Offline + Online]
    SI --> EMB[Embedding jobs]
    EMB --> VEC[Vector Index / Vector DB]
    VEC --> RAG[RAG / Search apps]

    UC[Catalog + Governance + Lineage] -.-> BR
    UC -.-> SI
    UC -.-> GO

    AG[AI Agents / NL2SQL tools] --> BI
    AG --> RAG

```

This diagram reflects a key reality of 2026: the lakehouse layers (bronze/silver/gold) remain central, but vectors and feature serving often become additional “satellite systems” for AI-native workloads. ²⁷

Part two: Architecture options and recommendations for an engineering manager

Current-state assumptions and goals

Assumptions (explicitly based on your prompt):

You have a **legacy relational DB** (system of record for transactional data) plus a **MongoDB-style document store** for project data. Scale is **unspecified**—so all sizing and some design choices must be validated via instrumentation (data volume growth, write rates, query concurrency, freshness/SLO requirements). You want an upgrade plan that improves analytics and enables AI-native access patterns (RAG/vector retrieval, NL2SQL, agent-friendly governance) while managing risk and avoiding a “big bang.”

Target outcomes (practical, measurable):

- A governed analytical “system of record” with open formats (to avoid lock-in and enable multi-engine compute). ²⁸
- Support batch + near-real-time ingestion via CDC/streaming (for fresher features and analytics). ²⁹
- Enable AI retrieval patterns: embeddings + vector search (warehouse-native or dedicated vector DB depending on SLOs). ³⁰
- Make the platform “AI-agent-safe”: strong lineage, least privilege, and cost controls to contain the risks of agentic query/code generation. ³¹

Comparison table of architecture families

Architecture family	What it is	Strengths	Weaknesses / watchouts	Best fit in AI era
Traditional lakehouse (OTF + object storage + distributed compute)	Open table formats on Parquet + engines like Spark/Trino + catalog	Open & interoperable (Iceberg works across engines); ACID-ish features on lakes (Delta/ Iceberg/ Hudi); good for large-scale analytics + ML training data ³²	Ops complexity if self-managed (catalog, compaction, governance); “serving latency” often needs extra systems; unstructured + vector retrieval requires add-ons ³³	Great as system of record + training data; pair with vector/ feature/ streaming serving layers

Architecture family	What it is	Strengths	Weaknesses / watchouts	Best fit in AI era
Cloud data warehouse-centric	Managed warehouse (Snowflake, BigQuery) as primary analytics store	Strong managed performance + governance; easy SQL consumption; now increasingly supports AI features (e.g., BigQuery vector search; Gemini query assistance; Snowflake VECTOR/Cortex Search) ³⁴	Cost can spike with unpredictable agent-generated queries; open-format interoperability depends on vendor features; external table performance varies ³⁵	Best when you want minimal ops and mostly SQL analytics + integrated AI search
“Open lake + warehouse compute” hybrid	Data stays in open formats (Iceberg/ Parquet) but multiple engines query it	Keeps data open; choose best engine per workload; BigLake unifies warehouse/lake access with fine-grained security on open formats like Iceberg/Parquet ³⁶	Governance and interoperability still require disciplined cataloging; cross-engine semantics drift; needs strong metadata and testing	Strong default for avoid-lock-in + multi-workload AI + analytics
Streaming-first (Kafka/ Flink-centric)	Event log is central; materialized views feed serving systems	Best for real-time features and stateful stream processing; Kafka Streams models streams as ordered, replayable, fault-tolerant sequences ³⁷	Harder for ad-hoc analytics unless you also land into lakehouse/ warehouse; operational complexity of state management	Essential when real-time is a requirement; usually complements lakehouse
Data mesh (org + architecture)	Domain teams publish governed “data products”	Improves scaling across org boundaries; Thoughtworks frames it as enabling independent teams to build reusable data products ³⁸	Not a single technology choice; requires operating model, governance, platform enablement; can fail without standards	Useful at org scale; pair with lakehouse/ warehouse foundations

Architecture family	What it is	Strengths	Weaknesses / watchouts	Best fit in AI era
Vector DB-centric for knowledge	Vector database is primary retrieval store	High-performance similarity search; purpose-built for RAG retrieval; Pinecone supports namespaces/partitioning; Milvus is a dedicated vector DB platform ³⁹	Not a full analytics system of record; still need warehouse/lakehouse for truth and batch processing	Use as retrieval tier, not the sole data platform

Open table formats and why they still matter

A key reason lakehouses remain relevant is that the “unit of durability” is shifting toward open table formats:

Apache Iceberg is explicitly an open table format for huge analytic datasets and integrates with multiple engines (Spark, Trino, Flink, etc.). ⁴⁰

Delta Lake provides ACID transactions and unifies batch and streaming processing on top of cloud storage. ⁴¹

Apache Hudi positions itself as an open data lakehouse platform with incremental processing and database-like functionality (upserts/deletes, concurrency control, table optimizations). ⁴²

Even warehouse vendors are adopting this layer:

Snowflake supports Apache Iceberg tables and documents that they use the Iceberg open table format specification, store data/metadata in external cloud storage, and can integrate with Snowflake Open Catalog for cross-engine access. ⁴³

Google BigLake is presented as a storage engine to unify warehouses and lakes and enable open formats like Iceberg and Parquet with fine-grained security without duplicating data. ⁴⁴

This convergence strongly suggests that lakehouses are not becoming obsolete; rather, the market is standardizing on *lakehouse-like storage semantics* even when the user experience is “warehouse.” ⁴⁵

Recommended concrete stack

Because your cloud/provider constraints are unspecified, the best recommendation is a “concrete but adaptable” stack: pick a primary managed platform for velocity, but anchor data in open table formats and adopt open catalogs/standards so you can change engines later.

Below is a recommended stack that is concrete (uses the tools you listed) and realistic for a small-to-mid platform team to run.

Layer	Recommended choice	Alternatives you should evaluate	Rationale	Effort / complexity
Storage format	Parquet	ORC (less common in mixed-engine stacks)	Parquet is the common columnar file substrate in lakehouses and is explicitly referenced as the file layer for many table formats ⁴⁶	Low
Table format (system of record)	Delta Lake on object storage with UniForm enabled (so Iceberg clients can read when needed)	Iceberg-first (if you want maximum multi-engine neutrality); Hudi-first (if upserts/streaming are dominant)	Delta provides ACID + batch/streaming unification; UniForm allows reading Delta tables with Iceberg clients by generating Iceberg metadata asynchronously ⁴⁷	Medium
Primary compute + orchestration	Databricks (Spark + managed workflows)	Trino for interactive SQL; Flink for advanced streaming	Databricks positions lakehouse for AI initiatives; Spark Structured Streaming unifies batch + streaming; Databricks reference architectures commonly use medallion layering ⁴⁸	Medium
Catalog + governance	Unity Catalog	Snowflake Open Catalog (Iceberg REST); Polaris (self-managed); AWS Glue/HMS	Unity Catalog is described as unified data and AI governance; it captures runtime lineage including column-level ¹³	Medium
Transformation / modeling	dbt (core + Semantic Layer where possible)	Dataform (GCP), native notebooks	Semantic layer centralizes metrics and reduces duplicated logic—especially important when AI generates SQL ⁴⁹	Medium

Layer	Recommended choice	Alternatives you should evaluate	Rationale	Effort / complexity
Streaming + CDC	Kafka + Debezium + Spark Structured Streaming	Managed CDC services; Flink	Debezium captures row-level DB changes into change event streams; Spark Structured Streaming can consume and process streams at scale ⁵⁰	Medium-High
Vector retrieval	Start with platform-native vector search (Databricks Mosaic AI Vector Search) for internal apps; add Pinecone or Milvus if SLOs demand	BigQuery VECTOR_SEARCH; Snowflake VECTOR/Cortex Search	Databricks vector search is integrated with governance/ productivity tools; Pinecone and Milvus are dedicated vector DBs; keep a path to dedicated vector infra if needed ⁵¹	Medium
Feature store	Feast (offline on lakehouse, online on low-latency store)	Vendor feature stores	Feast explicitly decomposes into offline store + online store + feature serving, matching common ML needs ⁵²	Medium
Lineage standardization	OpenLineage emission + platform lineage	Proprietary-only lineage	OpenLineage is an open standard for lineage metadata collection; helps avoid lineage silos ⁵³	Medium

Why not “warehouse-only” as the default recommendation? Because your data sources include a relational DB + a document db and you explicitly want lakehouse definitions/variants and AI-native patterns. A lakehouse core (open tables) keeps you interoperable across Snowflake Iceberg, BigLake, Trino, and other engines as needs evolve. ⁵⁴

Part two: Migration and infrastructure upgrade plan

Migration strategies to choose from

A dual-track approach is usually safest: build a new analytical/AI platform alongside legacy systems, migrate workloads incrementally, and keep the ability to fall back.

Strategy	How it works	Pros	Cons	When to use
Big bang replatform	Stop old analytics, move everything at once	Fast end-state	High risk; difficult rollback	Rarely justified
Strangler / incremental (recommended)	Stand up lakehouse core; ingest data continuously; migrate use cases one by one	Lowest risk; easy to prove value early	Requires temporary duplication; needs careful source-of-truth policy	Most teams
Dual-write	App writes to old DB and new platform	Can reduce lag	Increases app complexity; risk of divergence	Only when you must meet immediate new-SLO and can invest
CDC-based replication	Capture DB changes (Debezium) → stream → lakehouse	Keeps near-real-time sync; minimal app change	Requires event/stream ops; schema changes need discipline	Best for relational systems and many doc-store cases ¹⁷

Given your “legacy relational + MongoDB-style data,” CDC-based replication + a medallion lakehouse is typically the best backbone because it avoids app changes and supports near-real-time freshness. ⁵⁵

Data modeling guidelines for a lakehouse + AI workloads

For analytics (BI and reporting):

Gold layer: dimensional model (star schema) for stable business reporting, or “wide denormalized” tables for speed and simplicity; both are common in warehouse-style analytics.

Semantic layer: define shared metrics centrally (dbt Semantic Layer) so BI tools and NL2SQL agents rely on consistent definitions instead of reinventing joins/filters. ⁴⁹

For ML:

Offline features: derived from silver/gold tables (historical consistency is key). Feast explicitly separates offline (training) and online (serving) stores, which matches standard ML platform patterns. ⁵⁶

Online features: use a low-latency store (not the lakehouse) when you need real-time inference SLOs.

For RAG / embeddings:

Keep “source-of-truth” text and metadata in the lakehouse (silver/gold), then maintain embeddings + vector indexes either inside your platform (Databricks vector search; BigQuery vector search; Snowflake VECTOR/Cortex Search) or in a dedicated vector DB. ⁵⁷

Ingestion, storage formats, and catalog/metadata design

Ingestion:

Relational DB: Debezium CDC produces change events in order, allowing downstream consumers to build incremental tables. ⁵⁸

Streaming events/logs: Kafka Streams documentation emphasizes streams as ordered, replayable, fault-tolerant sequences; Spark Structured Streaming provides scalable stream processing using SQL-like expressions. ⁵⁹

Batch/file ingestion: land raw files into object storage as immutable “bronze” data; then transform into table-format-managed bronze tables.

Storage & table formats:

Use Parquet as the file format foundation. ⁶⁰

Use Delta Lake (or Iceberg/Hudi) as the table layer; Delta explicitly emphasizes ACID transactions and batch+streaming unification. ⁴¹

Enable time travel / history for rollback and auditing workflows. Databricks documents table version history and time travel for auditing/rollback use cases. ²⁰

Metadata/catalog:

Adopt a single “authoritative catalog” for tables, permissions, and lineage.

Unity Catalog is explicitly designed for unified governance for data and AI assets and captures runtime lineage. ¹³

If you need cross-engine Iceberg interoperability, evaluate Iceberg REST catalogs and Polaris. Iceberg defines a REST catalog spec, and Apache Polaris implements Iceberg’s REST API for multi-engine interoperability. ⁶¹

If you are Snowflake-centric but want open tables, Snowflake Open Catalog is built on the Iceberg REST protocol and is a managed service for Apache Polaris. ⁶²

Query patterns, latency, and cost tradeoffs you should design for

Interactive analytics SQL: medium latency (seconds) is usually fine; optimize using partitioning, clustering/Z-ordering strategies, and compaction in your table format (mechanisms vary for Delta/Iceberg/Hudi). ⁶³

Near-real-time analytics and feature freshness: streaming ingestion + incremental table updates; Spark Structured Streaming’s model supports continuous incremental updates. ⁶⁴

Vector retrieval for RAG: differentiate “index freshness” vs “query latency.” Snowflake notes Cortex Search’s primary use cases are RAG and enterprise search and provides low-latency query endpoints once a service is created; BigQuery supports vector search via VECTOR_SEARCH (optionally using a vector index). ⁶⁵

Cost control under AI-generated queries: NL2SQL tools and agents will create more exploratory queries. BigQuery's Gemini assistance can generate and fix queries, which is great—but it increases the need for guardrails (approved datasets/views, semantic layer, query governors, budgets). ⁶⁶

Phased timeline with milestones, risks, and rollback plans

The following plan assumes a small platform team and a “strangler” strategy. Adjust durations once you know scale, data volume, and stakeholder urgency.

```
gantt
  title Lakehouse modernization and AI-enablement timeline
  dateFormat YYYY-MM-DD
  axisFormat %b %Y

  section Foundation
  Inventory sources, SLAs, PII      :a1, 2026-04-06, 21d
  Choose table format + catalog    :a2, after a1, 14d
  Landing zone + IAM/KMS baseline  :a3, after a1, 21d

  section Ingestion (Bronze)
  CDC from relational (Debezium/Kafka) :b1, after a2, 35d
  Document ingestion (Mongo-style)    :b2, after a2, 35d
  Bronze tables + retention policies  :b3, after b1, 21d

  section Curation (Silver/Gold)
  Silver standardized models        :c1, after b3, 35d
  Gold marts + semantic layer (dbt) :c2, after c1, 35d
  BI cutover for first domain       :c3, after c2, 21d

  section AI-native capabilities
  Embeddings pipeline + vector index :d1, after c1, 35d
  Feature store (Feast) MVP          :d2, after c1, 35d
  RAG app pilot + evaluation         :d3, after d1, 28d

  section Hardening
  Monitoring + lineage + audits      :e1, after c1, 35d
  Cost controls + query guardrails   :e2, after c2, 21d
  Decommission legacy analytics jobs :e3, after c3, 60d
```

Key milestones and “definition of done” (DoD):

Foundation DoD: a chosen table format + catalog strategy; baseline security; documented data classification. Unity Catalog's unified governance model is an example of the goal state for “one place to manage permissions + discovery + lineage.” ¹³

Bronze DoD: continuous ingestion with replay capability. Debezium CDC provides ordered change event streams; Kafka Streams' replayable stream model supports rebuilding derived views when needed. ⁶⁷

Silver/Gold DoD: at least one domain fully modeled with tests, plus a semantic layer for core metrics. dbt Semantic Layer is purpose-built to define metrics and reduce duplicated join logic. ⁶⁸

AI-native DoD: a working embeddings + vector retrieval pipeline. BigQuery's VECTOR_SEARCH and Snowflake's VECTOR functions illustrate warehouse-native retrieval; Databricks Mosaic AI Vector Search illustrates lakehouse-native retrieval integrated with governance. ⁶⁹

Risks and rollback plans (per phase):

Foundation risks: wrong choice of primary table format or catalog; mitigate by choosing an approach that preserves interoperability (e.g., Delta + UniForm for Iceberg reads, or Iceberg + REST catalog), and by piloting with real workloads first. ⁷⁰

Rollback: keep legacy analytics running unchanged; treat the new platform as read-only until validated.

Bronze ingestion risks: schema drift and CDC operational issues. Mitigate with schema registry practices and automated compatibility checks; ensure streams are replayable (Kafka model) and transformations are idempotent (Spark Structured Streaming style). ⁵⁰

Rollback: stop downstream consumption; keep ingesting to raw storage; reprocess when stable.

Silver/Gold risks: semantic mismatch, stakeholder distrust, data quality issues. Mitigate with tests + lineage + incremental rollout by domain. Unity Catalog lineage helps trace what produced what. ⁷¹

Rollback: route BI back to legacy marts; keep gold tables versioned (time travel) to restore prior states. ²⁰

AI-native risks: retrieval quality regressions; security issues (prompt injection via retrieved text); cost spikes. Mitigate by sandboxing the AI app's data access, filtering retrieved context, and adopting OWASP/NCSC guidance that prompt injection is a distinct risk requiring defense-in-depth. ⁷²

Rollback: disable RAG features or revert to keyword search while keeping embeddings/index pipelines isolated.

Operational considerations for an AI-augmented data platform

Governance and security

Adopt "least privilege + audit everything" as non-negotiable because AI agents will broaden who can ask questions and what gets queried.

Unity Catalog is explicitly positioned as a unified governance solution and can capture runtime lineage across queries (including column-level), which is valuable for auditing and incident response. ¹³

If you need cross-engine governance for Iceberg tables, Snowflake Open Catalog provides an Iceberg REST catalog implementation with cross-engine security via RBAC (as described in Snowflake's getting started tutorial). ⁷³

For AI app security, treat prompt injection as a core architectural risk: NCSC recommends viewing prompt injection as exploiting an "inherently confusable deputy," and OWASP Top 10 for LLM Apps stresses that RAG

does not fully mitigate prompt injection. This implies you should constrain tool permissions, sanitize retrieved content, and isolate credentials used by agentic systems. ²⁶

CI/CD and infrastructure as code

Because AI coding agents can generate code quickly (Claude Code is explicitly described as reading your codebase, making changes across files, running tests, and delivering committed code), you should reinforce CI/CD gates rather than loosen them. ²⁵

Minimum recommended CI/CD practices:

Data transformations and models in version control with automated tests (schema tests, row count sanity, freshness checks).

Infrastructure as code for platform resources. For example, Databricks exposes Terraform resources for Mosaic AI Vector Search endpoints, enabling controlled rollouts and change reviews. ⁷⁴

Promotion environments (dev → staging → prod) with reproducible data snapshots, using table versioning/time travel to validate changes on stable datasets. ⁷⁵

Monitoring, lineage, and rollback mechanics

Monitoring should cover:

Pipeline health: lag, throughput, error rates (especially CDC). ⁷⁶

Table health: compaction/backlog, file counts, storage growth. (All three major table formats emphasize table services/optimizations; Hudi explicitly frames “table services” like rollbacks as part of operationalization.) ⁷⁷

Lineage: OpenLineage provides an open standard model (jobs, datasets, runs) to record lineage metadata during execution. ⁵³

Query cost + guardrails: especially important with NL2SQL and agentic usage. BigQuery’s Gemini assistance accelerates query writing, so you should pair it with guardrails (authorized views, semantic layer, budgets). ⁶⁶

Rollback strategies:

Table-level rollback: Databricks documents table history/time travel for rollback and auditing, while Hudi documents rollback behavior for handling partially failed writes. ⁷⁸

Pipeline rollback: keep streaming offsets/checkpoints and immutable bronze data so you can replay and rebuild silver/gold when needed (Kafka replay model + structured streaming incremental processing). ⁵⁹

Feature/vector rollback: treat indexes as derived artifacts; rebuild from source-of-truth tables (silver/gold) rather than trying to “patch” production indexes manually. This aligns with how warehouse-native vector search works (vectors stored in tables and searched via functions/indexes) and how vector DBs support bulk upserts. ⁷⁹

Tooling map for vectors, feature stores, and modern lakehouse variants

Vector tooling options (quick decision rubric):

Warehouse/lakehouse-native vectors: BigQuery VECTOR_SEARCH + optional vector index (good when embeddings live in tables and you want SQL-native similarity search). ⁸⁰

Snowflake VECTOR type + vector similarity functions; Snowflake also positions Cortex Search for RAG and enterprise search with low-latency endpoints. ⁸¹

Databricks Mosaic AI Vector Search (governance-integrated, built into Databricks). ⁸²

Dedicated vector DB: Pinecone supports “upsert text → vectors,” namespaces for multitenancy isolation, and hybrid dense/sparse patterns. ⁸³

Milvus is a dedicated vector database with its own deployment/ops model. ⁸⁴

Feature store: Feast’s explicit separation of offline store (historical extraction for training) and online store (low-latency serving) is a strong default conceptual model even if you later switch implementations. ⁵²

Lakehouse variants worth knowing about: BigLake: unifies warehouses and lakes and enables open formats like Iceberg/Parquet with fine-grained security. ⁴⁴

Snowflake Iceberg tables + Snowflake Open Catalog: keeps data in external storage but with managed catalog options and cross-engine access via REST catalog. ⁴³

Delta UniForm: allows delta tables to be read by Iceberg clients by generating Iceberg metadata asynchronously, helping mitigate lock-in concerns in Delta-first environments. ⁸⁵

Microsoft OneLake: positioned as a single place for all analytics data in Fabric; relevant if you’re in the Microsoft ecosystem and want a SaaS lakehouse experience. ⁸⁶

Bottom line recommendation

Traditional lakehouse architectures are still relevant—but the “complete solution” is no longer just “lakehouse + BI.” In 2026, the relevant design is:

Lakehouse core (open tables + governed catalog + batch/stream ingestion)

plus

AI-native serving layers (vector retrieval strategy, feature store, semantic layer, security guardrails for agentic access)

This approach aligns with (a) the industry consolidation around open table formats and open catalogs, (b) the expansion of native vector/AI features in warehouses and lakehouse platforms, and (c) the practical need to control risk (prompt injection, data exposure, runaway cost) when AI agents and NL2SQL become common entry points. ⁸⁷

¹ ⁶ Lakehouse: A New Generation of Open Platforms that Unify ...

https://www.vldb.org/cidrdb/2021/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics.html?utm_source=chatgpt.com

² ⁸ Retrieval-Augmented Generation for Large Language Models ...

https://arxiv.org/abs/2312.10997?utm_source=chatgpt.com

³ ²⁴ ⁴⁹ ⁶⁸ dbt Semantic Layer | dbt Developer Hub

https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl?utm_source=chatgpt.com

4 10 46 60 **Documentation - Apache Parquet**

https://parquet.apache.org/docs/?utm_source=chatgpt.com

5 34 **BigQuery overview | Google Cloud Documentation**

https://docs.cloud.google.com/bigquery/docs/introduction?utm_source=chatgpt.com

7 66 **Write queries with Gemini assistance | BigQuery - Google Cloud**

https://docs.cloud.google.com/bigquery/docs/write-sql-gemini?utm_source=chatgpt.com

9 81 **Vector data types - Snowflake Documentation**

https://docs.snowflake.com/en/sql-reference/data-types-vector?utm_source=chatgpt.com

11 14 41 47 **Welcome to the Delta Lake documentation | Delta Lake**

https://docs.delta.io/?utm_source=chatgpt.com

12 64 **Structured Streaming Programming Guide - Apache Spark**

https://spark.apache.org/docs/latest/streaming/index.html?utm_source=chatgpt.com

13 16 **What is Unity Catalog? | Databricks on AWS**

https://docs.databricks.com/aws/en/data-governance/unity-catalog/?utm_source=chatgpt.com

15 **Lakehouse connector — Trino 480 Documentation**

https://trino.io/docs/current/connector/lakehouse.html?utm_source=chatgpt.com

17 29 50 55 58 67 76 **Debezium Documentation**

https://debezium.io/documentation/reference/stable/index.html?utm_source=chatgpt.com

18 **Lakehouse reference architectures (download) | Databricks ...**

https://docs.databricks.com/aws/en/lakehouse-architecture/reference?utm_source=chatgpt.com

19 28 32 40 **Introduction - Apache Iceberg**

https://apache.github.io/iceberg/docs/latest/?utm_source=chatgpt.com

20 75 78 **Work with table history | Databricks on AWS**

https://docs.databricks.com/aws/en/delta/history?utm_source=chatgpt.com

21 **Lakehouse: A New Generation of Open Platforms that Unify ...**

https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf?utm_source=chatgpt.com

22 **レイクハウス リファレンス アーキテクチャ (ダウンロード ...**

https://learn.microsoft.com/ja-jp/azure/databricks/lakehouse-architecture/reference?utm_source=chatgpt.com

23 30 69 79 80 **Search embeddings with vector search | BigQuery - Google Cloud**

https://docs.cloud.google.com/bigquery/docs/vector-search?utm_source=chatgpt.com

25 **Claude Code | Anthropic's agentic coding system**

https://www.anthropic.com/product/claude-code?utm_source=chatgpt.com

26 72 **Prompt injection is not SQL injection (it may be worse)**

https://www.ncsc.gov.uk/blog-post/prompt-injection-is-not-sql-injection?utm_source=chatgpt.com

27 52 56 **Introduction | Feast: the Open Source Feature Store**

https://docs.feast.dev/?utm_source=chatgpt.com

31 71 **View data lineage using Unity Catalog | Databricks on AWS**

https://docs.databricks.com/aws/en/data-governance/unity-catalog/data-lineage?utm_source=chatgpt.com

33 Apache Hudi | An Open Source Data Lake Platform | Apache ...

https://hudi.apache.org/?utm_source=chatgpt.com

35 36 44 45 87 BigLake documentation | Google Cloud Documentation

https://docs.cloud.google.com/biglake/docs?utm_source=chatgpt.com

37 59 Core Concepts - Apache Kafka

https://kafka.apache.org/30/streams/core-concepts/?utm_source=chatgpt.com

38 Data mesh | Thoughtworks

https://www.thoughtworks.com/what-we-do/data-modernization/data-mesh?utm_source=chatgpt.com

39 83 Pinecone documentation - Pinecone Docs

https://docs.pinecone.io/guides/get-started/overview?utm_source=chatgpt.com

42 63 Overview | Apache Hudi

https://hudi.apache.org/docs/overview/?utm_source=chatgpt.com

43 Apache Iceberg™ tables - Snowflake Documentation

https://docs.snowflake.com/en/user-guide/tables-iceberg?utm_source=chatgpt.com

48 Data Lakehouse Architecture | Databricks

https://www.databricks.com/product/data-lakehouse?utm_source=chatgpt.com

51 57 82 Mosaic AI Vector Search - Databricks on AWS

https://docs.databricks.com/aws/en/vector-search/vector-search?utm_source=chatgpt.com

53 About OpenLineage | OpenLineage

https://openlineage.io/docs/?utm_source=chatgpt.com

54 62 Snowflake Open Catalog overview | Snowflake Documentation

https://docs.snowflake.com/en/user-guide/opencatalog/overview?utm_source=chatgpt.com

61 REST Catalog Spec - Apache Iceberg™

https://iceberg.apache.org/rest-catalog-spec/?utm_source=chatgpt.com

65 Cortex Search - Snowflake Documentation

https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-search/cortex-search-overview?utm_source=chatgpt.com

70 85 Universal Format (UniForm) | Delta Lake

https://docs.delta.io/delta-uniform/?utm_source=chatgpt.com

73 Getting started with Snowflake Open Catalog | Snowflake ...

https://docs.snowflake.com/en/user-guide/opencatalog/tutorials/open-catalog-gs?utm_source=chatgpt.com

74 databricks_vector_search_endpoint | Resources | databricks ...

https://registry.terraform.io/providers/databricks/databricks/1.110.0/docs/resources/vector_search_endpoint?utm_source=chatgpt.com

77 Auto Rollbacks | Apache Hudi

https://hudi.apache.org/docs/rollbacks/?utm_source=chatgpt.com

84 Milvus vector database documentation

https://milvus.io/docs?utm_source=chatgpt.com

86 Microsoft OneLake documentation - Microsoft Fabric

https://learn.microsoft.com/en-us/fabric/onelake/?utm_source=chatgpt.com